C10 · G4 Hackathon

Multi-Agent DevOps Incident Analysis Suite

From raw logs to a reviewed incident plan, a Jira ticket and a Slack alert in about 15 seconds.

LangGraph FastAPI OpenRouter ChromaDB Slack Jira

The problem

On-call engineers lose the first 30 minutes of every incident to manual triage.

🔎 Needle in a haystack

Thousands of log lines across services; the root cause hides in a handful.

✍️ Repetitive paperwork

Writing the Slack update and Jira ticket by hand while the system is down.

📚 Tribal knowledge

Runbooks exist, but nobody finds the right one at 3 a.m.

⚠️ Automation risk

Fully automatic bots spam channels, leak secrets and file duplicate tickets.

Solution overview

Upload a log → six specialised agents analyse it → a human approves what gets sent.

1 · Understand

Parse `.log / .json / .csv`, redact secrets, strip prompt-injection lines, group events into incidents with severity and evidence.

2 · Recommend

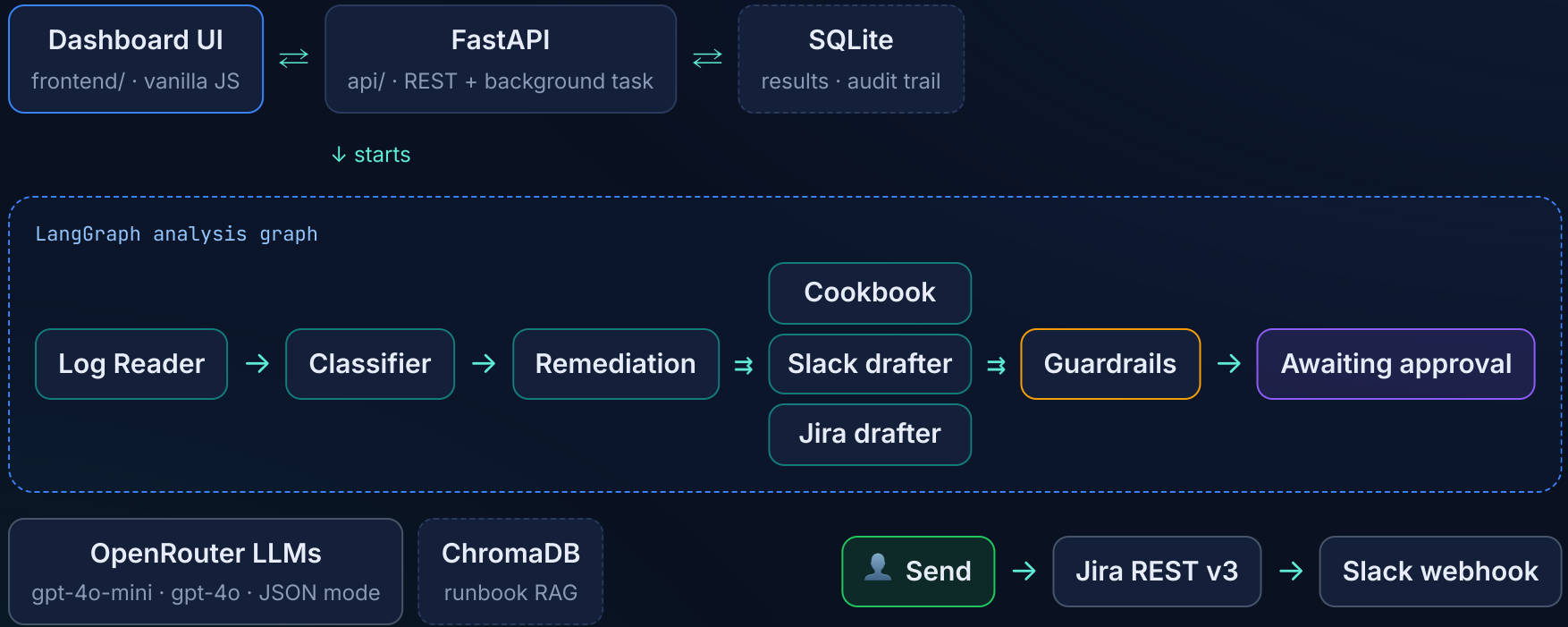
Retrieve matching runbooks (RAG) and produce grounded fix plans, a rollback plan and one ordered checklist.

3 · Notify safely

Draft Slack alerts and Jira tickets. **Send** or **Don't send** per critical issue — every decision audited.

~15 s	$0.06	0.96	100%
per analysis (was 34.5 s)	LLM cost per run	category recall (evals)	evidence grounding

Multi-agent architecture



Agent responsibilities

Agent	Input → Output	Model	Fallback if the LLM fails
Log Reader	raw file → parsed, redacted events	none (deterministic)	n/a
Classifier	events → incidents + severity + evidence IDs	`gpt-4o-mini`	one incident per failing service
Remediation	incident + retrieved runbooks → fix plan, rollback, risk	`gpt-4o`	steps from best-matching runbook
Cookbook	all plans → one ordered checklist	`gpt-4o`	merged, de-duplicated actions
Slack drafter	incidents → Slack mrkdwn alert	`gpt-4o-mini`	message template
Jira drafter	ERROR/CRITICAL incidents → ticket drafts	`gpt-4o`	ticket template

LangGraph workflow

```python
# orchestration/graph.py
def build_analysis_graph():
    graph = StateGraph(AnalysisState)
    graph.add_node("parse_logs", traced("Log Reader")(parse_logs))
    graph.add_node("classify_incidents", traced("Classifier")(...))
    graph.add_node("generate_remediation", traced("Remediation")(...))
    for name in DRAFTERS:              # cookbook, slack, jira
        graph.add_node(name, traced(...)(...))
    graph.add_node("apply_guardrails", traced("Guardrails")(...))

    # early exit when there is nothing to analyse
    graph.add_conditional_edges("parse_logs", _after_parse,
                                ["classify_incidents", END])
    graph.add_conditional_edges("classify_incidents", _after_classify,
                                ["generate_remediation", END])
    for name in DRAFTERS:                                  # fan out
        graph.add_edge("generate_remediation", name)
    graph.add_edge(DRAFTERS, "apply_guardrails")           # fan in
    graph.add_edge("apply_guardrails", "await_approval")
    return graph.compile()
```

```python
class AnalysisState(AnalysisResult):
    # parallel agents append,
    # never overwrite each other
    agent_runs: Annotated[
        list[AgentRun], operator.add]
    guardrail_findings: Annotated[
        list[GuardrailFinding], operator.add]
```

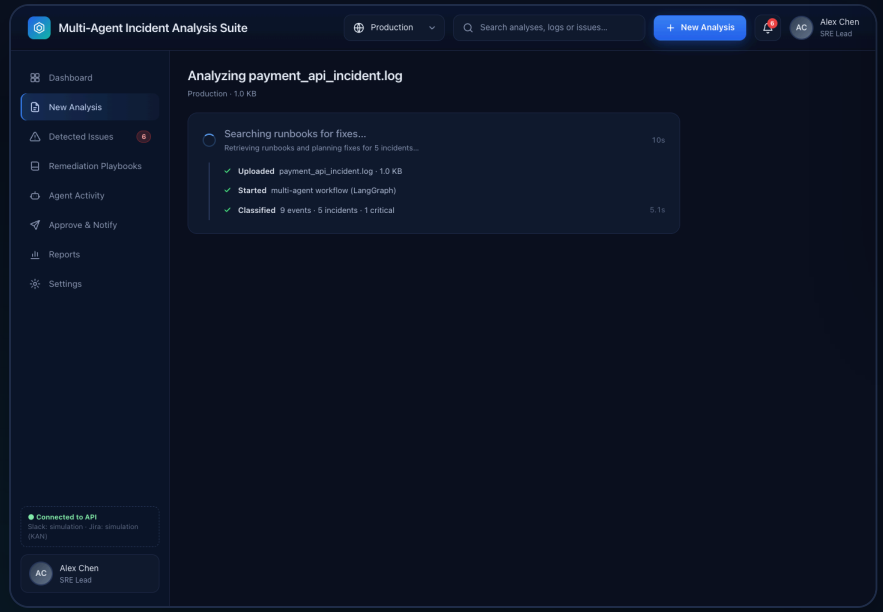
Why it matters

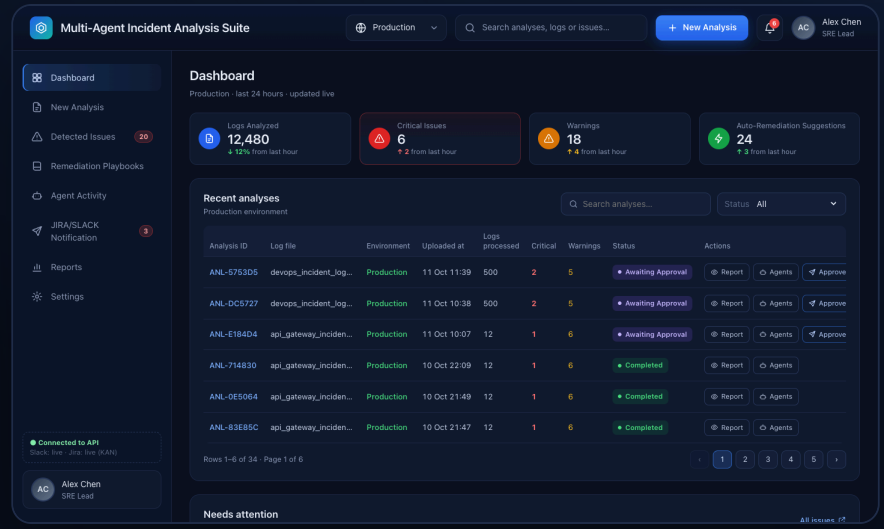
- Early exit when a log has no incidents
- Parallel drafters: **34.5 s → 15.5 s**
- Typed Pydantic state, validated at every hop
- `@traced` captures latency, tokens, cost

UI walkthrough · 1

New Analysis — upload, then *Start Analysis* sits right under the drop zone.

While agents work, one LLM-style status line shows what's happening now; finished steps tick off below.

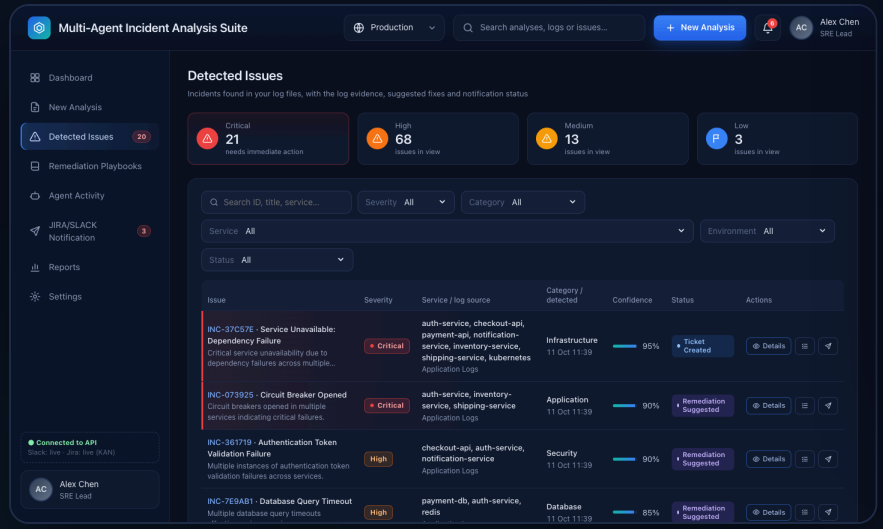


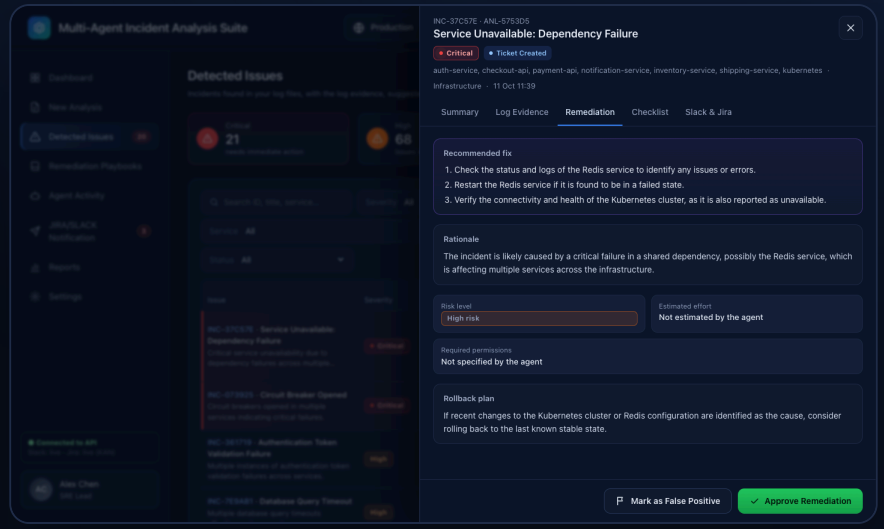


Dashboard — compact KPI row, recent analyses with pagination and filters, and the open critical/high issues that need attention.

UI walkthrough · 2

Detected Issues — severity tiles, filters, and per-issue evidence.





Issue drawer — Summary · Log Evidence · Remediation (runbooks cited, risky steps flagged ⚠️) · Checklist · Slack & Jira.

Example incident scenario

```
# sample_logs/order_processing_incident.csv
12:01:02 ERROR    order-api  Insert failed: deadlock
12:01:03 WARNING  order-db   Long-running txn (45 s)
12:01:04 ERROR    order-api  HTTP 500 returned
12:01:05 CRITICAL order-api  Queue depth > 10000
12:01:08 ERROR    order-api  rabbitmq:5672 refused
12:01:10 ERROR    shipping   Shipping API 504
```

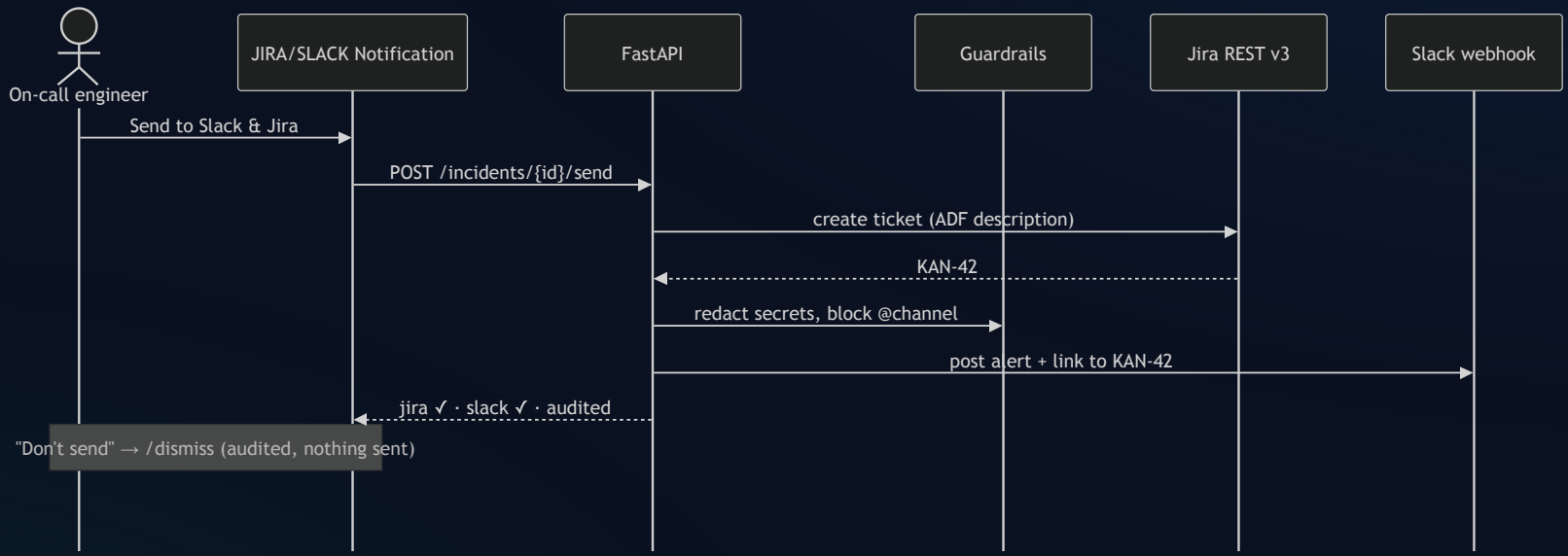
```
{
  "title": "Order Processing Queue Depth Exceeded",
  "severity": "CRITICAL",
  "category": "PERFORMANCE",
  "affected_services": ["order-api"],
  "evidence_event_ids": ["evt-0009", "evt-0010"],
  "confidence": 0.9
}
```

Agent output · 14 s · $0.06

- **7 incidents**, 1 critical: *Order Processing Queue Depth Exceeded*
- Runbooks matched: queue backlog, database deadlock, third-party dependency
- Checklist: 29 ordered steps (Do now / Investigate / Prevent)

Evidence IDs must exist in the log — anything the model invents is dropped (grounding = 1.0 in evals).

Slack & Jira integration flow



Simulation ↔ Live

Switch per integration in Settings. Default: Simulation.

Idempotent retry

Reuses the ticket, never double-posts to Slack.

Secrets server-side

Webhook and token never reach the browser.

Business benefits

⏱️ Faster MTTR

Triage drops from ~30 minutes of reading logs to ~15 seconds of analysis plus a review.

🧠 Runbooks actually used

RAG puts the team's own documented fix in front of the engineer, with citations.

🛡️ Safe automation

Human approval, secret redaction, injection screening and destructive-step flags — all audited.

💸 Predictable cost

~$0.06 per analysis; cheap model for classification, stronger model only where reasoning matters.

📈 Measurable quality

Eval suite gates releases: recall, grounding, severity accuracy, latency and cost budgets.

🔁 Consistent comms

Every Slack alert and Jira ticket follows the same structure: cause, impact, actions, rollback.

Technology stack

Orchestration & AI

LangGraph · OpenRouter · gpt-4o · gpt-4o-mini · JSON mode · Pydantic

Retrieval & data

ChromaDB · all-MiniLM-L6-v2 · SQLite (WAL)

Backend

FastAPI · BackgroundTasks · pydantic-settings · rate limit · API key

Frontend

Vanilla JS SPA · SVG charts · Streamlit (alt UI)

Integrations

Slack webhook · Jira REST v3 + ADF

Quality & ops

pytest · 43 tests · LLM-judge evals · Docker · JSON logs · /health /ready

```
make install && make env      # venv + .env from .env.example
make api                      # http://localhost:8000/ui/
make test && make eval        # 43 tests · eval gate
```

Next steps

Scale

- Postgres + task queue (Celery / Arq) + Redis rate limits
- Streaming log ingestion (CloudWatch, Loki, Kafka)
- SSO and real approver identity in the audit trail

Smarter agents

- More runbooks: deadlocks, brokers, auth, third-party APIs
- Feedback loop: misclassified incidents become eval cases
- Correlate with deploys and metrics, not only logs

Integrations

- Slack interactive buttons (approve from Slack)
- Jira assignee mapping and status sync
- PagerDuty / Opsgenie escalation for criticals

Try it

`github.com/MdFasikhan/C10_G4_Hackathon`

Thank you

Questions?

C10 · G4 — Multi-Agent DevOps Incident Analysis Suite